
APACHE AIRFLOW 3.1.7



STAGE 3 / INTEGRATION

Integration

Kafka, Spark, Cassandra, REST API & Custom Operator

Target:	Backend / Distributed Systems Engineer
Duration:	Ngày 6-9 (2-3 giờ/ngày)
Focus:	External system integration
Version:	Apache Airflow 3.1.7

Mục lục

1. Tổng quan tích hợp

2. Kafka Integration Patterns

2.1. Pattern A: Airflow orchestrate Kafka → Spark

2.2. Pattern B: Không dùng Airflow làm consumer

3. Cassandra/DSE Integration

4. Custom Operator cho E-invoice XML Validation

5. HTTP/REST API Integration (mTLS + httpx)

6. Bài tập thực hành

1. Tổng quan tích hợp

Stage 3 tập trung vào việc kết nối Airflow với các hệ thống bên ngoài. Trong môi trường production thực tế, Airflow không hoạt động độc lập mà là trung tâm điều phối (orchestrator) cho nhiều service khác nhau:

- **Kafka** — Stream data, kiểm tra lag, trigger batch jobs
- **Spark** — Xử lý dữ liệu lớn, chạy qua SparkSubmitOperator
- **Cassandra/DSE** — Lưu trữ dữ liệu, verify kết quả ghi
- **REST API** — Đồng bộ dữ liệu với hệ thống bên ngoài qua HTTP/mTLS

Mục tiêu của Stage 3 là bạn viết được DAG thực tế, tích hợp đa hệ thống, có xử lý lỗi, retry, và verify kết quả.

2. Kafka Integration Patterns

Airflow không phải streaming consumer. Nó là batch orchestrator. Vì vậy, pattern đúng khi tích hợp với Kafka là: **Airflow kiểm tra điều kiện trên Kafka, rồi trigger batch job.**

2.1. Pattern A: Airflow orchestrate Kafka → Spark

Đây là pattern phổ biến nhất: Airflow kiểm tra Kafka lag, nếu đủ lớn thì submit Spark job để xử lý:

```

# dags/stage3_kafka_spark.py
from airflow import DAG
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

with DAG(
    'kafka_to_spark_invoice_etl',
    schedule=timedelta(hours=1),
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage3', 'kafka', 'spark'],
) as dag:

    def check_kafka_lag(**context):
        """Kiểm tra Kafka lag – trong production dùng KafkaAdminClient."""
        import json
        lag_info = {"topic": "invoices", "lag": 1500, "threshold": 1000}
        if lag_info['lag'] < lag_info['threshold']:
            raise Exception("Not enough new messages, skipping")
        context['ti'].xcom_push(key='lag_info', value=json.dumps(lag_info))
        return lag_info

    check_lag = PythonOperator(
        task_id='check_kafka_lag',
        python_callable=check_kafka_lag,
    )

    spark_job = SparkSubmitOperator(
        task_id='spark_invoice_transform',
        application='/opt/spark/jobs/invoice_etl.py',
        conn_id='spark_default',
        conf={
            'spark.executor.memory': '2g',
            'spark.executor.cores': '2',
        },
        application_args=[
            '--date', '{{ ds }}',
            '--topic', 'invoices',
            '--output', 'redis://redis:6379/2/invoices/{{ ds }}'
        ],
    )

    def verify_cassandra_load(**context):
        """Verify dữ liệu đã được ghi vào Cassandra."""
        from cassandra.cluster import Cluster
        cluster = Cluster(['cassandra-node1', 'cassandra-node2'])
        session = cluster.connect('invoice_db')
        result = session.execute(
            "SELECT COUNT(*) FROM invoices WHERE date = %s",
            [context['ds']]
        )

```

```
print(f"Loaded {count} records to Cassandra for {context['ds']}")
assert count > 0, "No records found!"

verify = PythonOperator(
    task_id='verify_cassandra',
    python_callable=verify_cassandra_load,
)

check_lag >> spark_job >> verify
```

Bảng 1: Luồng dữ liệu trong pattern Kafka → Spark → Cassandra

Task	Công việc	Công nghệ
check_kafka_lag	Kiểm tra dữ liệu mới trên Kafka topic	KafkaAdminClient / JMX metrics
spark_invoice_transform	Submit Spark job xử lý batch	SparkSubmitOperator
verify_cassandra	Confirm data được ghi thành công	cassandra-driver

2.2. Pattern B: Không dùng Airflow làm consumer

Anti-pattern: Dùng Airflow làm real-time consumer

Airflow **không phải** streaming consumer. Nếu cần real-time (< 1 giây), hãy dùng **Spark Streaming**, **Flink**, hoặc **Kafka Streams**. Airflow chỉ nên orchestrate batch reconciliation.

Các trường hợp nên dùng và không nên dùng:

Bảng 2: Khi nào dùng Airflow với Kafka

Tình huống	Dùng Airflow?	Giải pháp thay thế
Batch xử lý mỗi giờ/ngày từ Kafka	Có	Airflow + Spark batch
Real-time < 1 phút	Không	Spark Streaming, Flink
Reconciliation sau streaming	Có	Airflow daily + Spark
Event-driven trigger	Có (với Asset)	Airflow 3.x Asset triggers

3. Cassandra/DSE Integration

Airflow cung hook sẵn cho Cassandra thông qua provider package. Hook này quản lý connection pool và retry.

```
from airflow.providers.apache.cassandra.hooks.cassandra import CassandraHook

def cassandra_upsert(**context):
    """Ghi dữ liệu vào Cassandra sử dụng Airflow Hook."""
    hook = CassandraHook(cassandra_conn_id='cassandra_default')
    hook.get_conn().execute(
        "INSERT INTO invoices (id, date, amount, status) VALUES (%s, %s, %s, %s)",
        (context['run_id'], context['ds'], 100.0, 'PROCESSED')
    )
```

Cài đặt Connection (UI hoặc CLI):

```
airflow connections add 'cassandra_default' \
    --conn-type 'cassandra' \
    --conn-host 'cassandra-node1,cassandra-node2' \
    --conn-port '9042' \
    --conn-extra '{"keyspace": "invoice_db", "load_balancing_policy":
"TokenAwarePolicy"}'
```

Bảng 3: Tham số Connection Cassandra

Tham số	Ý nghĩa	Ví dụ
conn-type	Loại connection	cassandra
conn-host	Danh sách node	cassandra-node1,cassandra-node2
conn-port	CQL port	9042
conn-extra	JSON config thêm	keyspace, load_balancing_policy, protocol_version

4. Custom Operator cho E-invoice XML Validation

Custom Operator cho phép đóng gói logic nghiệp vụ tái sử dụng. Dưới đây là operator validate hóa đơn điện tử XML — kiểm tra các trường bắt buộc theo quy định:

```

# plugins/operators/xml_validator_operator.py
from airflow.models.baseoperator import BaseOperator
from airflow.exceptions import AirflowException
import xml.etree.ElementTree as ET
from pathlib import Path

class XMLValidationOperator(BaseOperator):
    """
    Custom operator kiểm tra hóa đơn điện tử XML.
    Kiểm tra các trường bắt buộc: KHMSHDon, KHHDDon, SHDon, NBan/MST.
    """
    template_fields = ('xml_path', 'schema_path')

    def __init__(self, xml_path: str, schema_path: str, **kwargs):
        super().__init__(**kwargs)
        self.xml_path = xml_path
        self.schema_path = schema_path

    def execute(self, context):
        self.log.info(f"Validating {self.xml_path} against {self.schema_path}")

        xml_file = Path(self.xml_path)
        if not xml_file.exists():
            raise AirflowException(f"XML file not found: {self.xml_path}")

        try:
            tree = ET.parse(xml_file)
            root = tree.getroot()

            # Validate required tags
            required_tags = ['KHMSHDon', 'KHHDDon', 'SHDon', 'NBan']
            for tag in required_tags:
                elem = root.find(f'://{tag}')
                if elem is None or not elem.text:
                    raise AirflowException(f"Missing or empty tag: {tag}")

            # Validate MST (Tax code) format
            mst = root.find('://NBan/MST')
            if mst is None or len(mst.text) < 10:
                raise AirflowException("Invalid MST format")

            self.log.info("XML validation passed")
            return {"file": self.xml_path, "valid": True, "mst": mst.text}

        except ET.ParseError as e:
            raise AirflowException(f"XML parse error: {e}")

```

Cách cấu trúc Custom Operator

1. Kế thừa từ `BaseOperator` hoặc `BaseSensorOperator`
2. Định nghĩa `template_fields` để hỗ trợ Jinja templating
3. Implement `__init__` để nhận tham số
4. Implement `execute()` — đây là logic chính
5. Dùng `AirflowException` để báo lỗi, task sẽ tự động fail
6. Đặt file trong `plugins/`, restart Airflow để load

5. HTTP/REST API Integration (mTLS + httpx)

Trong hệ thống e-invoice thực tế, cần gọi API bên ngoài với xác thực mTLS (mutual TLS). Airflow 3.x hỗ trợ điều này thông qua `PythonOperator` kết hợp với thư viện `httpx`.


```

# dags/stage3_api_integration.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import httpx
import ssl

with DAG(
    'api_invoice_sync',
    schedule=timedelta(minutes=30),
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage3', 'api', 'mtls'],
) as dag:

    def sync_invoices(**context):
        """Đồng bộ hóa hóa đơn qua API với mTLS authentication."""
        cert_path = '/certs/client.crt'
        key_path = '/certs/client.key'
        ca_path = '/certs/ca.crt'

        context_ssl = ssl.create_default_context(cafile=ca_path)
        context_ssl.load_cert_chain(cert_path, key_path)

        with httpx.Client(
            http2=True,
            verify=context_ssl,
            timeout=30.0
        ) as client:
            response = client.post(
                'https://api.einvoice.gov.vn/sync',
                json={"date": context['ds'], "batch_size": 1000}
            )
            response.raise_for_status()
            return response.json()

    sync_task = PythonOperator(
        task_id='sync_via_mtls',
        python_callable=sync_invoices,
    )

```

Bảng 4: So sánh HTTP clients cho Airflow

Thư viện	Ưu điểm	Nhược điểm	Phù hợp
<code>httpx</code>	HTTP/2, async, mTLS tốt	Thêm dependency	API hiện đại, mTLS
<code>requests</code>	Quen thuộc, đơn giản	Không hỗ trợ HTTP/2	API đơn giản
<code>airflow.providers.http</code>	Tích hợp sẵn, retry	Ít linh hoạt	GET/POST đơn giản

6. Bài tập thực hành

Bảng 5: Bài tập Stage 3

STT	Bài tập	Kỹ năng
1	Tạo Custom Operator XMLValidationOperator và dùng trong DAG	Viết custom operator
2	Setup Airflow Connection cho Cassandra, test hook	Quản lý connections
3	Viết DAG gọi SparkSubmitOperator (dùng Spark local mode)	Tích hợp Spark
4	Tạo DAG gọi REST API với httpx + mTLS (self-signed cert)	API + security
5	Tạo Plugin với custom Hook cho Kafka AdminClient	Viết custom hook
6	Test <code>airflow tasks test <dag_id> <task_id> <date></code> cho từng task	Debug từng task

Quy tắc viết Plugin

File plugin đặt trong thư mục `plugins/`, phải restart Airflow để load. Cấu trúc: `plugins/operators/` cho operators, `plugins/hooks/` cho hooks, `plugins/sensors/` cho sensors. Trong DAG, import như module thường: `from operators.xml_validator_operator import XMLValidationOperator`.